

# POKEMON STRENGTH CLASSIFIER

Technical Analysis Report

Classifying Pokemon Power Rankings using Artificial Neural Networks (MLP)

|           |                                  |
|-----------|----------------------------------|
| Algorithm | MLPClassifier (sklearn)          |
| Dataset   | PokemonDatabase.csv              |
| Classes   | 8 tiers — G, F, E, D, C, B, A, S |
| Samples   | 202 Pokemon                      |
| Split     | 70% train / 15% dev / 15% test   |

## 1. Project Overview

The goal of this project is to build a machine learning model that classifies Pokemon into 8 power tiers (G through S) based on their combat statistics. The problem is a multi-class classification task: the model must learn how stats like HP, Attack, Defense, and Speed map to a competitive tier.

### 1.1 Label Encoding

| Label | Encoding | Description                                 |
|-------|----------|---------------------------------------------|
| G     | 0        | Lowest tier — weakest Pokemon               |
| F     | 1        | Very weak tier                              |
| E     | 2        | Weak tier                                   |
| D     | 3        | Below average tier                          |
| C     | 4        | Average tier                                |
| B     | 5        | Above average tier                          |
| A     | 6        | Strong tier                                 |
| S     | 7        | Highest tier — legendary / mythical Pokemon |

## 2. Data — PokemonDatabase.csv

### 2.1 Input Features

| Feature Name         | Meaning          | Notes                        |
|----------------------|------------------|------------------------------|
| Health Stat          | Hit Points (HP)  | Survivability                |
| Attack Stat          | Physical Attack  | Normal move damage           |
| Defense Stat         | Physical Defense | Reduces normal damage        |
| Special Attack Stat  | Special Attack   | Special move damage          |
| Special Defense Stat | Special Defense  | Reduces special damage       |
| Speed Stat           | Speed            | Determines who attacks first |
| Base Stat Total      | Total Base Stats | Sum of all stats above       |

## 3. Data Preprocessing

### 3.1 Standard Scaling

StandardScaler is applied before training to normalize all features to zero mean and unit variance. Without this step the network would be dominated by large-magnitude features (Base Stat Total can exceed 700) while lower-range features like Speed (5–180) contribute very little. Gradient descent also converges faster on

normalized inputs.

### 3.2 PCA Variant

A second dataset was produced by applying PCA after scaling, reducing the 7 input dimensions to 2 principal components. This enables 2D decision boundary visualization and removes collinearity between Base Stat Total and the individual stats.

## 4. Model Architecture — MLPClassifier

MLPClassifier (Multi-Layer Perceptron) is a feedforward neural network where every neuron in a layer is connected to every neuron in the next. It was chosen because it can learn non-linear decision boundaries between tiers — something a linear model cannot — while remaining straightforward to configure.

### 4.1 Default Configuration

| Parameter          | Value  | Purpose                                           |
|--------------------|--------|---------------------------------------------------|
| hidden_layer_sizes | (10,)  | 1 hidden layer, 10 neurons                        |
| activation         | relu   | Passes positive values unchanged, zeros negatives |
| alpha              | 0.0001 | L2 regularization — penalizes large weights       |
| max_iter           | 1000   | Max optimization iterations                       |
| random_state       | 42     | Fixed seed for reproducibility                    |

### 4.2 Network Layout

- Input layer → 7 neurons (one per feature)
- Hidden layer → 10 neurons with ReLU activation
- Output layer → 8 neurons with Softmax (one per tier)

## 5. Hyperparameter Experiments — Standard Dataset

A grid search was conducted over  $\alpha_{\text{power}}$ ,  $\text{max\_iter\_power}$ , and  $\text{hidden\_layer\_amt}$ , where  $\alpha = 10^{\alpha_{\text{power}}}$  and  $\text{max\_iter} = 10^{\text{max\_iter\_power}}$ . Dev accuracy is the primary selection criterion.

| $\alpha_{\text{power}}$ | $\text{max\_iter\_power}$ | hidden | Fit?     | Train Acc | Dev Acc | F1   |
|-------------------------|---------------------------|--------|----------|-----------|---------|------|
| -4                      | 2                         | 10     | underfit | 0.606     | 0.569   | 0.52 |
| -4                      | 3                         | 10     | good     | 0.734     | 0.693   | 0.67 |
| -4                      | 4                         | 10     | good     | 0.735     | 0.698   | 0.67 |
| -1                      | 2                         | 10     | underfit | 0.605     | 0.569   | 0.52 |
| -1                      | 3                         | 10     | good     | 0.743     | 0.698   | 0.67 |
| -1                      | 4                         | 10     | good ★   | 0.744     | 0.703   | 0.68 |
| 0                       | 2                         | 10     | underfit | 0.604     | 0.569   | 0.51 |
| 0                       | 3                         | 10     | good     | 0.731     | 0.678   | 0.65 |
| 0                       | 4                         | 10     | good     | 0.729     | 0.673   | 0.63 |
| 1                       | 2                         | 10     | underfit | 0.537     | 0.540   | 0.46 |
| 1                       | 3                         | 10     | underfit | 0.650     | 0.639   | 0.56 |
| 1                       | 4                         | 10     | underfit | 0.650     | 0.639   | 0.56 |
| -1                      | 3                         | 20     | overfit  | 0.784     | 0.678   | 0.65 |
| -1                      | 3                         | 50     | overfit  | 0.807     | 0.708   | 0.69 |
| -1                      | 3                         | 100    | overfit  | 0.818     | 0.698   | 0.68 |

★ Selected as ideal:  $\alpha=0.1$ ,  $\text{max\_iter}=1000$ ,  $\text{hidden}=10 \rightarrow$  dev accuracy 70.3%

### Key Observations

- Accuracy improves as  $\text{max\_iter}$  increases but plateaus around  $10^3$ .
- Reducing  $\alpha$  below 0.1 yields no meaningful gain.
- $\alpha$  has almost no effect when  $\text{max\_iter}$  is low ( $10^2$ ) — enough iterations are needed for regularization to act.
- With 10 hidden neurons the train–dev gap stays below 0.05, indicating a clean fit.
- Increasing hidden neurons raises train accuracy but widens the train–dev gap — the model overfits.
- $\text{hidden\_layer\_amt}=50$  achieves 70.8% dev accuracy but at a 0.099 gap; not selected.

## 6. Hyperparameter Experiments — PCA Dataset

The same grid search was repeated on the 2-component PCA dataset. The PCA model generally tracked the standard model closely — until hidden layer size was pushed to 100.

### 6.1 Notable Result

With `hidden_layer_sizes=(100,1)`, `alpha=0.0001`, and `max_iter=500`, the PCA model reached a dev accuracy of 0.7227 — higher than any standard-dataset run. Train accuracy was 0.7394, giving a train–dev gap of only 0.017, which indicates a good fit rather than overfitting.

| Tier         | Samples | Precision | Recall | F1-Score |
|--------------|---------|-----------|--------|----------|
| G            | 26      | 0.92      | 0.92   | 0.92     |
| F            | 55      | 0.93      | 1.00   | 0.96     |
| E            | 41      | 0.71      | 0.90   | 0.80     |
| D            | 16      | 0.30      | 0.19   | 0.23     |
| C            | 14      | 0.27      | 0.29   | 0.28     |
| B            | 20      | 0.48      | 0.60   | 0.55     |
| A            | 17      | 0.88      | 0.41   | 0.56     |
| S            | 13      | 0.60      | 0.23   | 0.33     |
| Weighted Avg | 202     | 0.72      | 0.72   | 0.70     |

PCA model — `hidden=(100,1)`, `alpha=0.0001`, `max_iter=500` | Train: 0.739 Dev: 0.7227

## 6.2 Comparison: Standard vs PCA

| Criterion            | Standard Model | PCA Model (best) |
|----------------------|----------------|------------------|
| Input dimensions     | 7 features     | 2 PCA components |
| Best dev accuracy    | 0.703          | <b>0.7227</b>    |
| Train accuracy       | 0.744          | 0.739            |
| Train–dev gap        | 0.041          | <b>0.017</b>     |
| Fit assessment       | Good fit       | Good fit         |
| F tier recall        | —              | 1.00             |
| E tier recall        | —              | 0.90             |
| Mid-tier (D/C) perf. | Poor           | Still poor       |

- PCA achieves a marginally better dev accuracy (0.7227 vs 0.703) with a tighter train–dev gap.
- The top-volume tiers (F, E) improve notably under PCA, suggesting the principal components capture a cleaner separation for high-population classes.
- Mid-tiers (D, C) remain problematic under both models — insufficient samples and overlapping stat ranges are the limiting factor, not the dimensionality.

## 7. Problems Identified

---

### Class Imbalance

F tier has 55 samples; S tier has only 13 — a 4.2× ratio. The model is biased toward majority classes and consistently underperforms on minority tiers regardless of architecture.

### Insufficient Data for 8 Classes

~202 samples across 8 classes averages ~25 per class. Both the standard and PCA models struggle with D, C, A, and S tiers because there simply is not enough signal to learn reliable boundaries.

### Tier Boundary Overlap (D / C / B / A)

Mid-tier Pokemon have heavily overlapping stat distributions. A Pokemon with BST ~480 could plausibly sit in C, B, or A depending on meta context — raw stats alone are not sufficient to disambiguate.

### Feature Multicollinearity

Base Stat Total is the arithmetic sum of the other six stats. Including both introduces redundant information, adds noise, and can mislead the weight updates during training.

### Overfitting at High Capacity

Pushing the standard model to larger hidden layers (e.g. 100 neurons) widens the train–dev gap to 29.4%. The PCA model at the same depth avoids this, likely because the compressed representation acts as implicit regularization.

## 8. Recommended Solutions

---

The following solutions address the root causes identified above. Where results from applying a solution are available within this project, the outcome is reported directly.

### 8.1 Fix Class Imbalance — SMOTE / class\_weight

Apply SMOTE to synthetically oversample minority tiers (D, C, A, S), or pass `class_weight='balanced'` to the classifier.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X_train, y_train)
```

*Effect: Not yet tested within this project. Expected to be the single highest-impact improvement — the per-tier classification report consistently shows near-zero recall on tiers with fewer than 16 samples, which oversampling directly addresses.*

### 8.2 Switch to Random Forest or XGBoost

Tree-based ensembles handle class imbalance more gracefully than MLP and expose feature importance scores that help diagnose which stats are actually driving tier predictions.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(
    n_estimators=200, class_weight='balanced', random_state=42 )
```

*Effect: Not yet tested within this project. Likely to outperform MLP on this dataset size given that tree ensembles are less sensitive to the small-sample-per-class problem.*

### 8.3 Apply PCA to Reduce Collinearity

Reducing the 7 features to 2 PCA components removes the multicollinearity between Base Stat Total and the individual stats. This was tested in Section 6.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
```

*Effect: Confirmed improvement. The PCA model with hidden=(100,1) reached dev accuracy 0.7227 vs 0.703 for the best standard-dataset run — a +1.97pp gain — with a tighter train-dev gap (0.017 vs 0.041). F and E tier recall improved substantially.*

### 8.4 Add Richer Features

Pokemon Type, Legendary flag, Generation number, and evolution stage all carry tier-relevant information that base stats alone cannot express.

```
# Features to add: # 'Type 1', 'Is Legendary', 'Generation', 'Evo Stage'
```

*Effect: Not yet tested. Expected to help most for mid-tiers where stat overlap is the primary limitation — type and legendary status are strong differentiators there.*

### 8.5 Stratified K-Fold Cross Validation

The fixed 70/15/15 holdout gives a single noisy estimate on ~30 dev samples per class. StratifiedKFold(n\_splits=5) gives 5 estimates that are averaged for a more stable result.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring='f1_macro')
```

*Effect: Not yet tested. Will not improve the model itself but will produce more reliable accuracy estimates, which is important given the small dataset size.*

### 8.6 Merge Tiers or Use Ordinal Classification

If data remains too sparse after oversampling, merging D+C and A+S into 6 tiers increases samples per class. An ordinal classifier can also exploit the natural ordering of the tiers.

```
tier_merge = {'G':0, 'F':1, 'E':2, 'D':3, 'C':3, 'B':4, 'A':5, 'S':5}
```

*Effect: Not yet tested. Merging is a trade-off — simpler boundaries but coarser predictions. Ordinal regression could recover some ordering signal without full merging.*

## 9. Overall Results Summary

| Model                                     | Train Acc | Dev Acc | Gap   | Status     |
|-------------------------------------------|-----------|---------|-------|------------|
| Standard — alpha=-1, iter=3, h=10 (ideal) | 0.744     | 0.703   | 0.041 | Good fit ★ |
| Standard — h=50 (best dev)                | 0.807     | 0.708   | 0.099 | Overfit    |
| Standard — h=100 (large)                  | 0.818     | 0.698   | 0.120 | Overfit    |
| PCA — h=(100,1), alpha=-4, iter=500       | 0.739     | 0.7227  | 0.017 | Good fit ★ |

★ Both the ideal standard model and the best PCA model maintain a tight train–dev gap. The PCA model edges ahead on dev accuracy while also being less prone to overfitting at higher capacities — making it the stronger candidate for further development.

### Remaining ceiling:

Both models still struggle with mid-tiers D, C, A, and S due to class imbalance and overlapping stat distributions. The next meaningful accuracy jump will require either synthetic oversampling (SMOTE), richer features (type, legendary flag), or a switch to tree-based models — none of which have been tested yet.



## 10. Test Set Evaluation

After selecting the best model from each approach on the dev set, both were evaluated on the held-out test set (203 samples). The test set was not used during training or model selection — this is the first time either model has seen these Pokemon.

### 10.1 Test Accuracy Overview

| Model                                   | Dev Acc | Test Acc | Drop   | Verdict     |
|-----------------------------------------|---------|----------|--------|-------------|
| Standard — alpha=0.1, iter=1000, h=10   | 0.703   | 0.6995   | -0.004 | Consistent  |
| PCA — h=(100,1), alpha=0.0001, iter=500 | 0.7227  | 0.6847   | -0.038 | Slight drop |

The standard model is the more reliable of the two on unseen data — its dev and test scores are nearly identical (0.703 vs 0.700), confirming the dev result was not a lucky split. The PCA model drops 3.8pp from dev to test, suggesting it fit the dev split slightly more tightly.

### 10.2 Per-Tier Accuracy on Test Set

| Tier | Samples | Standard correct | Standard acc | PCA correct | PCA acc | Harder for |
|------|---------|------------------|--------------|-------------|---------|------------|
| G    | 27      | 25               | 0.93         | 26          | 0.96    | Standard   |
| F    | 56      | 55               | 0.98         | 55          | 0.98    | —          |
| E    | 42      | 41               | 0.98         | 36          | 0.86    | PCA        |
| D    | 16      | 2                | 0.12         | 1           | 0.06    | Both       |
| C    | 13      | 7                | 0.54         | 8           | 0.62    | Standard   |
| B    | 20      | 6                | 0.30         | 9           | 0.45    | Standard   |
| A    | 16      | 5                | 0.31         | 2           | 0.12    | PCA        |
| S    | 13      | 1                | 0.08         | 2           | 0.15    | Standard   |

- G, F: both models perform well — large sample sizes, clear stat separation.
- E: standard model dominates (98% vs 86%). PCA loses separation in 2D for this tier.
- D: both models fail almost completely (6–12%). Only 16 samples and heavy overlap with C/E.
- B: PCA does better (45% vs 30%) — PCA components appear to capture B-tier better in 2D.
- A, S: both models struggle. Standard edges out on A (31% vs 12%); PCA edges out on S (15% vs 8%).

### 10.3 Error Analysis

| Model    | Total errors | Adjacent (off by 1) | Far (off by 2+) | Worst single error           |
|----------|--------------|---------------------|-----------------|------------------------------|
| Standard | 61           | 39 (64%)            | 22 (36%)        | Pikachu: S → F (6 tiers off) |
| PCA      | 64           | 44 (69%)            | 20 (31%)        | Pikachu: S → F (6 tiers off) |

The majority of errors in both models are adjacent misclassifications — the model predicts a tier one step away from the correct one. This is expected given the continuous nature of stat distributions across tiers. Far errors (off by 2 or more) are more concerning and are concentrated in A and S tier Pokemon with unusual stat profiles

— Pikachu (S tier due to Pikachu-specific meta rules) and Cosmog (B tier despite very low base stats) are clear outliers that raw stats cannot capture.

#### **10.4 Verdict**

The standard model generalizes more consistently to unseen data (test accuracy 69.95%) and is the safer production choice. The PCA model performs better on B-tier and S-tier predictions but drops on E and A tiers — its 2D compression loses information that the full 7-feature model retains. Both models confirm that the dev-set results were not overly optimistic: the real performance ceiling with current features and data is approximately 70%.